Cloud Virtualization - GSLC

1. Jelaskan alur lengkap perjalanan interupsi I/O saat sebuah physical device menerima data hingga signal tersebut diteruskan menjadi virtual interupt bagi Guest OS!

   1) Perangkat fisik menerima / menyelesaikan I/O → signal interupsi ke CPU / APIC host.
   2) Hypervisor menangkap interupsi fisik, karena Hypervisor yang mengontrol semua sumber daya perangkat keras → memeriksa konteks data / vektor interupsi untuk menentukan target VM
   3) Hypervisor mengsimulasikan interupsi dengan inject virtual interupt ke dalam interrupt controller virtual milik VM target.
   4) Pada saat target VM sedang dijadwalkan (scheduled) berjalan di pCPU, virtual interrupt controller virtual akan memicu interupsi ke Guess OS, sehingga driver perangkat di dalam VM dapat menyelesaikan I/O.

2. Apa yang terjadi jika VM penerima sedang dalam kondisi not scheduled pada pCPU? Mengapa penangan interupsi I/O yang kurang efisien dapat memicu overhead tinggi pada hypervisor? Sebutkan dampak teknisnya terhadap throughput I/O dan penggunaan CPU host!

   - Jika VM target not scheduled pada pCPU, Hypervisor tidak dapat segera menyuntikan virtual interupt karena konteks eksekusi VM tidak aktif.
     - Pending State : Hypervisor akan menandai status virtual interrupt "pending" dalam data internal VM.
     - Wake-up Trigger : Hypervisor akan memicu mekanisme wake-up / mengubah prioritas jadwal VM.

   - Penangan interupsi I/O yang tidak efisien mengakibatkan overhead tinggi karena setiap interupsi memicu VM Exit

   - Dampak teknis:
     - Throughput I/O : Jalur I/O menjadi lebih panjang karena melibatkan Guest OS dan Hypervisor. Jika Hypervisor melakukan banyak emulasi, latensi pemrosesan data / paket meningkat drastis dan menurunkan throughput I/O.
     - Penggunaan CPU host : Setiap VM Entry dan Exit mengonsumsi sirklus CPU host untuk memuat / menyimpan status register dan table page. Jika interupsi tinggi, CPU host akan sibuk untuk konteks switching virtualisasi, bukan memproses data aktual.

3. Bandingkan mekanisme Hypervisor dalam mengontrol akses I/O antara Port-Mapped I/O (PMIO) dan Memory-Mapped I/O (MMIO)!

Dipindai dengan CamScanner

Fitur	PMIO	MMIO
Interuksi Akses	Menggunakan instruksi khusus I/O : IN & OUT.	Menggunakan instruksi standar LOAD (MOV) & STORE.
Mekanisme Trap	Hypervisor mengkonfigurasi bit pada register kontrol prosesor.	Hypervisor memanfaatkan manipulasi table page untuk membuat memory page.
Granulitas	Biasanya per port address (16 / 32 bit).	Per memory page (~4Kb).
kinerja	Cenderung lebih lambat. karena instruksi khusus dan sterilisasi pipeline ketat.	Secara teori lebih cepat pada native, namun pada virtualisasi sering kali lebih mahal.

A. Pada MMIO, bagaimana hypervisor memanfaatkan manipulasi tabel halaman memori (Extended Page Tables / EPT) untuk menjebak (trap) instruksi LOAD / STORE saat VM mengakses register perangkat I/O?

1. VM akses register via GPA, tapi Hypervisor tidak memetakan langsung ke HPA perangkat.
2. Halaman GPA diatur tidak valid / RW dilarang → Setiap LOAD/STORE memicu EPT Violation.
3. CPU keluar ke Hypervisor → Hypervisor baca instruksi, akses / perangkat / emulasi, lalu return result.
4. VM lanjut → Akses selalu melalui trap, makannya overhead I/O tinggi jika sering.

Dipindai dengan CamScanner